

Efficient Data Storage Systems

(Sistemas de Armazenamento de Dados Eficientes)

Storage Benchmarking



Systems Benchmarking

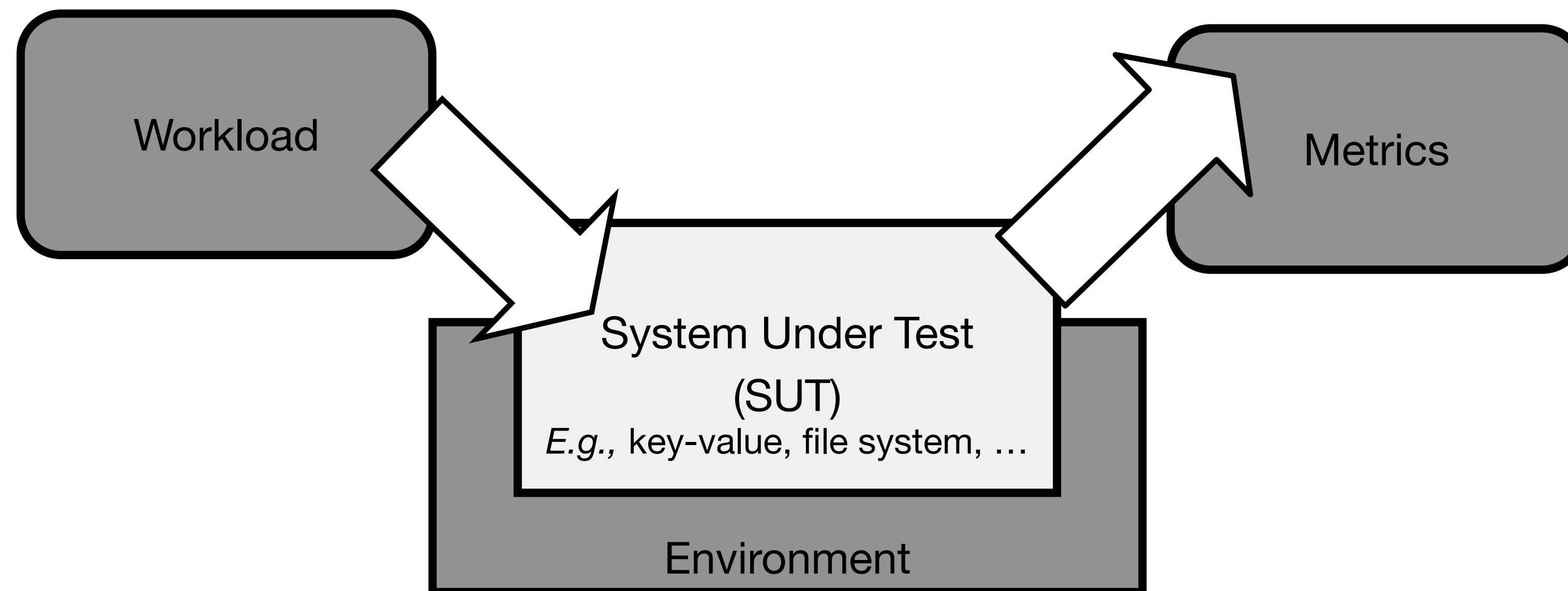
Why is it useful? (recap...)

- How can one build efficient and reliable (storage) systems?
 - Follow good design and implementation guidelines
 - Review and validate the code and implementation (e.g., formal verification)
 - **Test** (i.e., **benchmark**) implementations!
- How can one know that the system meets the desired requirements?
 - **Benchmark** the system!

Storage Systems Benchmarking

Ecosystem

- A Benchmarking ecosystem consists of
 - The **workload** (group of requests) is being issued to the **System Under Test (SUT)**
 - The **environment** where the SUT is running
 - The **metrics** collected to measure the performance, efficiency, and/or reliability of the SUT



Workload

Realistic workloads

- Testing storage systems in **production** may not be viable, or the best option
 - Imagine only knowing whether your file system actually stores user data after deployment and while serving users...
- One option is to use **traces of requests** from real workloads (e.g., from users or applications using a given storage system)
 - Finding traces and scaling these without losing realism is not easy...
 - Take a look at <http://iotta.snia.org/traces> for some examples of traces
- Another option is to use **real applications**
 - For example, running a database (MySQL, Postgres) or an AI framework (PyTorch, TensorFlow) over the storage system
 - The problem is choosing workloads to run over these applications that make their requests to the storage system realistic...

Workload

Synthetic workloads

- ◎ A final option is to use **synthetic workloads**, which are useful
 - When the aforementioned options are not viable
 - To evaluate specific functionalities of the SUT (e.g., performance serving random write requests)
 - To stress the SUT and/or test its scalability (*i.e.*, increasing the number of clients)
- ◎ Synthetic benchmarks may need to support multiple configurations to mimic distinct behaviors found in real production settings
 - **Interfaces** (e.g., key-value, file, block, ...)
 - **Operations** (e.g., read, write, stat, mkdir, unlink, ...)
 - **Access patterns** (e.g., sequential, random, Zipfian, ...)
 - But that's not all...

Workload

Synthetic workloads

- There are other properties of the workload that may need to be configured carefully to reproduce realistic evaluation scenarios
 - **Size** of data being read/written
 - **Parallelism** (*i.e.*, number of clients)
 - Workloads with **mixed operations** (*e.g.*, reads and writes, metadata)
 - **Rate** of client requests (*i.e.*, **stress testing** vs controlled client throughput)
- And even more tricky characteristics, such as
 - **Content** of data being written (*e.g.*, important for assessing deduplication)
 - Temporal and spatial **locality** of requests (*e.g.*, important for assessing caches)
 - ...

Synthetic Benchmarks

Some examples

- **FIO**: Used for testing **block/file-based data I/O operations**
 - <https://github.com/axboe/fio>
- **Filebench**: Used for evaluating **file system data and metadata I/O operations**
 - <https://github.com/filebench/filebench>
- **DEDISbench**: Used for testing deduplication systems **with realistic content for data I/O operations**
 - <https://github.com/jtpaulo/dedisbench>
- **DFS-Perf**: Used for evaluating the performance of various **distributed file systems**
 - <http://pasa-bigdata.nju.edu.cn/dfs-perf/>
- **YCSB**: Used for testing **NoSQL databases** and **key-value stores**
(**Note**: several projects are adapting their own YCSB version)
 - <https://github.com/brianfrankcooper/YCSB/>
- **MLPerf Storage**: Used for testing **AI workloads' storage requests**
 - <https://github.com/mlcommons/storage>
- and many more...

Environment

Hardware and software

- ◎ Knowing and setting up the right environment to run experiments is very important!
 - Testing your SUT in an unrealistic environment may lead to **wrong conclusions**
 - It is also important for others to be able to **reproduce** your results
- ◎ One must be able to **characterize the experimental environment**
- ◎ Hardware
 - What disk models are being used?
 - And CPU, RAM, network, and GPU? These may impact the workloads!
 - What hardware configurations are being used? (e.g., number of CPUs, amount of RAM, ...)
- ◎ Software
 - What operating system is being used? And the kernel version?
 - What about the libraries and corresponding versions?
 - And finally, what are the versions of the different SUT components?

Metrics

Understanding your SUT

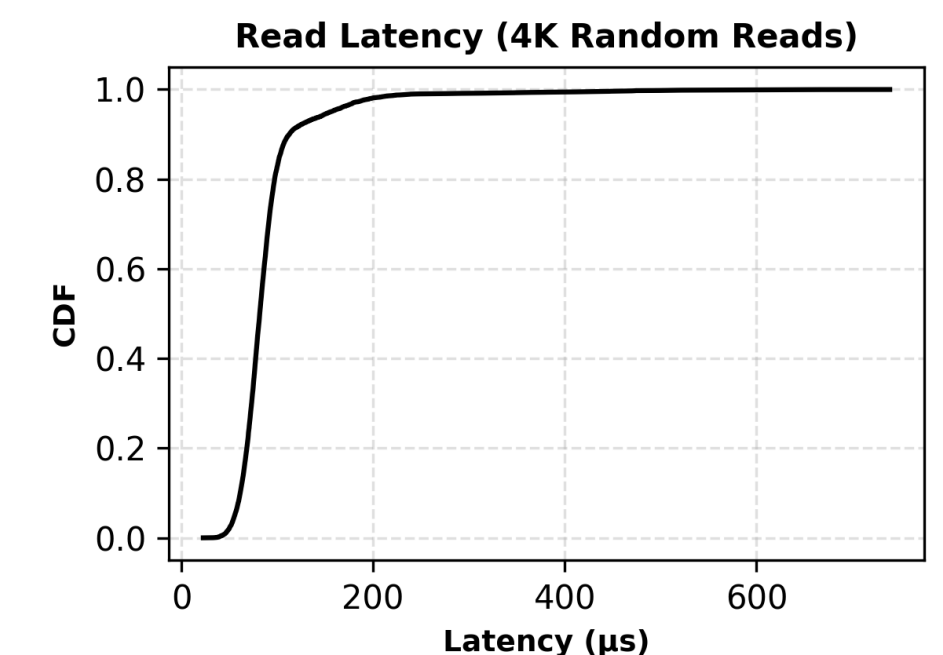
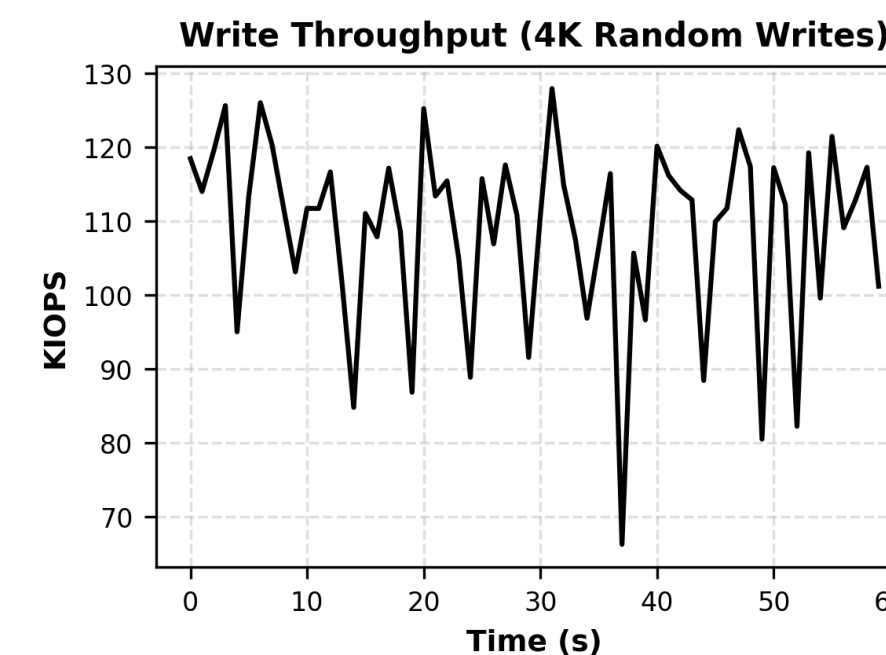
- **Performance** of the system
(e.g., latency, throughput of operations¹)
- **Utilization** of resources
(e.g., CPU, RAM, network, disk, energy overhead)
- **Efficiency** of the system
(e.g., the ratio between performance and resource usage)
- **Reliability** and **Availability** of a system
(e.g., number of errors, failures, uptime)
- ...

¹A naive view (latency = 1 / throughput) is false! This is only true when the system is busy 100% of the time executing exactly 1 request

Analysis and Representation

Of collected metrics

- When running experiments, one must collect several **samples** for metrics of interest!
 - Repeat your experiments several times to identify **abnormal behavior** (outliers)
 - Remember that many workloads **may not be deterministic!**
 - The behavior of the SUT and the underlying OS stack (e.g., the OS) may not be deterministic either!
- After collecting the samples, these must be **analyzed** and, ideally, **summarized**
 - Mean, standard deviation (std. dev.), percentiles, confidence intervals, ...
- Use **visual representations** to observe and better understand your samples!
 - Time-series, histograms, Empirical Cumulative Distribution Function (ECDFs), ...



Useful tools

Metric collection and analysis

● Metric collection

- Most benchmarks already report some metrics (e.g., throughput, latency, ...)
- Use tools, such as *dstat*, *iostat*, ... for resource monitoring
 - Or more advanced ones (e.g., *Beats* + *ElasticSearch*, *Prometheus*)

● Analysis

- Scripts! (*Python* is nice for this...)
- *Gnuplot* for plots!
- Or more advanced visualization tools such as *Kibana* and *Graphana*

Common Mistakes

When benchmarking storage systems

- No goals or biased ones
 - Define the **evaluation questions** you want to answer with your experiments!
 - First, ask the questions...
- Unrepresentative benchmarks
 - Choose **realistic workloads, metrics**, and experiments that help answer your questions
 - Avoid bias by only showing workloads, environments, and metrics that perfectly fit your solution. Don't be afraid to show **strengths** and **drawbacks**!
- Compare your solution with other ones
 - For example, with **baseline setup(s)** to show the benefits/drawbacks of your design and optimizations
 - Or other **related solutions** (i.e., the state of the art!)
 - Again, **be fair** in your comparison!

Common Mistakes

When benchmarking storage systems

- ◎ Unsystematic approach - **Reproducibility is key!**
 - Set and document your workloads, experimental environment, and configurations!
 - Be sure that your environment is set the same way for each experiment
- ◎ Software and hardware may exhibit distinct behavior across experiments
 - For instance, due to caches¹, background tasks (e.g., SSD's GC), ...
- ◎ Wrong analysis and presentation of results (i.e., misunderstanding measurements)
 - **Inspect samples in time** for stability
 - Consider, again, external phenomena (e.g., cache warmups, garbage collection)
 - Use appropriate **summarization techniques** and **visual representations**

¹ The OS page cache, for instance, may be dropped before running each experiment for increased reproducibility across experiments

More Information

- R. Jain. *The Art of Computer Systems Performance Analysis*. Wiley, 1991.
 - Chapters 1 to 6 and 9 to 13
- Gernot Heiser. ***Systems Benchmarking Crimes***.
<https://gernot-heiser.org/benchmarking-crimes.html>
- Tarasov et al. ***Benchmarking File System Benchmarking: It *IS* Rocket Science***.
HotOS, 2011.
- SNIA. ***Storage Performance Benchmarking Guidelines - Part I: Workload Design***. 2010
<https://www.snia.org/sites/default/files/PerformanceBenchmarking.Nov2010.pdf>

Questions?